



ULM UNIVERSITY OF APPLIED SCIENCES
FACULTY OF COMPUTER SCIENCE

Thesis Title

A thesis submitted in partial fulfillment of the requirements for the degree of
Master of Science in Computer Science

by

Mohamad Arabi

Computer Science

Faculty of Computer Science, Ulm University of Applied Sciences, 2026

April

2026

Supervised by

Prof. Dr. Traub

Prof. Dr. [Co-supervisor]

Ulm, 2026

Statement

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Mohamad Arabi

Signature

.....

Date: 15 April, 2026

Abstract

Security operations teams face alert overload and limited context, while most SIEM systems remain rule-centric and weak in reasoning. This thesis introduces a generic SIEM reaction plugin that operationalizes reasoning through an intelligence layer composed of explainable scoring, decision planning, policy constraints, and analyst feedback. The architecture cleanly separates orchestration from intelligence: raw alerts are normalized and enriched, scored using a transparent risk function with context multipliers and top-reason attributions, then passed to a planner that generates structured response plans. Plans are validated against policy rules and executed through constrained action adapters, with all steps recorded in an audit log. Analyst feedback and investigation traces are captured to update model weights and to recommend next investigation steps, closing the cognitive loop.

The system is implemented as a single runnable project, and a demo interface is provided that exposes raw alerts, normalized evidence, risk reasoning, plans, policy checks, and execution outcomes. Evaluation on synthetic and public security datasets demonstrates robust planning quality and end-to-end traceability, showing how reasoning-driven response can reduce analyst effort while preserving safety and interpretability. This work contributes a practical, extensible foundation for next-generation SIEM workflows that move beyond static correlation toward adaptive, explainable decision support.

Summary

Acknowledgment

I would like to express my deepest appreciation to all those who contributed to the successful completion of my master's thesis. Foremost, I extend my heartfelt appreciation to my esteemed advisor, Prof. Dr.-Ing. Traub. His guidance, insights, counsel, and mentorship were instrumental in shaping this research.

I also thank Prof. [Second Examiner], who graciously served as the second examiner. Lastly, I would like to thank all individuals and resources, too numerous to name, who contributed indirectly through their research, books, articles, and online sources.

Mohamad Arabi
Department of Computer Science
Technische Hochschule Ulm
Ulm, Germany
15 April, 2026

Table of Contents

Abstract	iii
Summary	iv
Acknowledgment	v
Table of Contents	vi
List of Figures	x
List of Algorithms	xi
List of Abbreviations	xii
List of Symbols	xiii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	2
1.4 Thesis Structure	2
2 Background and Related Work	3
2.1 Cybersecurity Monitoring and Security Operations Centers	3
2.2 Security Information and Event Management (SIEM)	3
2.2.1 Log Collection and Normalization	3
2.2.2 Event Correlation	3
2.2.3 Alerting and Incident Response	3
2.3 Limitations of Current SIEM Systems	3
2.3.1 Rule-Based Detection	3
2.3.2 Alert Overload and False Positives	3
2.3.3 Scalability and Data Volume	3
2.3.4 Dependence on Human Analysts	3
2.4 Research Gap	3
3 System Requirements	4
3.1 Functional Requirements	4
3.2 Non-Functional Requirements	4
3.3 Security and Safety Requirements	5

4	Design and Architecture Decisions	6
4.1	Design Goals	6
4.2	Architectural Style Selection	6
4.3	Separation of Intelligence and Orchestration Layers	6
4.4	Reasoning and Explainability Design	6
4.5	Dataset Pipeline Design	6
4.6	Safety and Fallback Mechanisms	6
5	System Architecture	7
5.1	System Overview	7
5.2	Event Ingestion Layer	8
5.3	Normalization and Enrichment	9
5.4	Threat Scoring Component	10
5.5	Decision Planning Component	11
5.6	Policy and Safety Layer	12
5.7	Action Execution and Response	12
5.8	Audit and Explainability Components	13
5.9	Cross-Cutting Concerns	14
6	Implementation	16
6.1	Technology Stack	16
6.1.1	Orchestration Layer: .NET 9 / C#	16
6.1.2	Intelligence Layer: Python / FastAPI	17
6.1.3	Model Training: PyTorch and scikit-learn	17
6.1.4	Persistence	17
6.1.5	Configuration	18
6.2	Core Engine Implementation	18
6.2.1	The Orchestration Pipeline	18
6.2.2	Interface-Driven Architecture	19
6.2.3	Domain Model: Alert Lifecycle	20
6.2.4	Normalization and the Multi-SIEM Mapping Registry	21
6.2.5	Enrichment Pipeline	21
6.2.6	Webhook Mode and Polling Mode	22
6.3	Intelligence Service	22
6.3.1	Service Startup and Backend Initialisation	23
6.3.2	Request Handling and the LRU Cache	23
6.4	Model Registry and Profile-Based Backend Selection	24
6.5	Threat Scoring Component	26
6.5.1	The Classifier Scorer	26
6.5.2	Enrichment-Aware Confidence Boosting	27
6.5.3	Hypothesis and Evidence Generation	28
6.6	Decision Planning Component	28
6.6.1	Framing Planning as Structured Prediction	29
6.6.2	The Custom Neural Network Planner	29
6.6.3	Feature Engineering	30
6.6.4	Multi-Head Training with Masked Action Loss	31
6.6.5	Enrichment Post-Processing Layer	32

6.7	Policy and Safety Layer	33
6.7.1	Three-Tier Decision Model	33
6.7.2	Entity Presence Validation	35
6.7.3	Action Catalog	35
6.8	Action Execution and Response	35
6.9	Audit, Traceability, and Console Output	36
6.9.1	Demo Trace Writer	37
6.9.2	Console Output	37
6.10	Deployment and System Setup	38
6.10.1	Single-Command Startup	38
6.10.2	Webhook Demo Setup	38
6.10.3	Environment Configuration Reference	39
7	Dataset Pipeline and Training	40
7.1	Security Datasets and Their Roles	40
7.1.1	MITRE ATT&CK (STIX)	40
7.1.2	Mordor / Security Datasets	41
7.1.3	CIC-IDS2018	41
7.1.4	DARPA Transparent Computing (TC)	42
7.1.5	Synthetic Scenario Generation	42
7.2	Dataset Processing and Normalisation	43
7.2.1	Schema Mapping	43
7.2.2	Label Assignment and Severity Calibration	44
7.2.3	Action Balancing	45
7.3	Training Data Format Evolution	45
7.3.1	Version 1: Full JSON Prompt	46
7.3.2	Version 2: Canonical Flat Format	46
7.3.3	Version 3: Compact Pipe-Delimited Format (Final)	46
7.4	The Scorer: From Non-Functional Baseline to Calibrated Classifier	47
7.4.1	Attempt 1: Zero-Shot LLM (baron-security)	47
7.4.2	Attempt 2: QLoRA Fine-Tuning on Foundation-Sec-8B	48
7.4.3	Attempt 3: TF-IDF Classifier Pipeline (Final)	48
7.5	The Planner: Four Approaches to Structured Prediction	49
7.5.1	Attempt 1: Zero-Shot LLM	50
7.5.2	Attempt 2: Foundation-Sec-8B QLoRA Fine-Tuning	50
7.5.3	Attempt 3: Seq2Seq Fine-Tuning (flan-t5)	50
7.5.4	Attempt 4: Custom Multi-Head MLP (Final)	52
7.6	Evaluation and Results Comparison	53
7.7	Integration with the Intelligence Service	54
8	Evaluation	56
8.1	Evaluation Setup	56
8.2	Detection Performance	56
8.3	Response Planning Evaluation	56
8.4	Example Incident Scenarios	56
8.5	Discussion of Results	56

9	Discussion	57
9.1	Advantages of the Proposed System	57
9.2	Comparison with Traditional SIEM Approaches	57
9.3	Limitations	57
9.4	Threats to Validity	57
10	Conclusion and Future Work	58
10.1	Summary of Contributions	58
10.2	Future Research Directions	58
	References	59
A	Appendix	60
A.1	Additional Figures	60
A.2	Configuration Details	60

List of Figures

5.1	High-level system decomposition showing the two-process boundary.	8
-----	---	---

List of Algorithms

List of Abbreviations

SIEM	S ecurity I nformation and E vent M anagement
SOAR	S ecurity O rchestration, A utomation, and R esponse
SOC	S ecurity O perations C enter
EDR	E ndpoint D etection and R esponse
XDR	eX tended D etection and R esponse
IOC	I ndicator o f C ompromise
TTP	T actics, T echniques, and P rocedures
ATT&CK	A dversarial T actics, T echniques, and C ommon K nowledge
LLM	L arge L anguage M odel
ML	M achine L earning
JSON	J ava S cript O bject N otation
API	A pplication P rogramming I nterface
STIX	S tructured T hreat I nformation eX pression
TAXII	T ruste d A utomated eX change of I ndicator I nformation

List of Symbols

R	risk score (overall)
b	bias term in risk model
w_i	feature weight i
x_i	feature value i
m	context multiplier
c	confidence (0–1)
S	severity (0–100)
P	response priority (0–100)

Introduction

1.1 Motivation



Security operations centers rely on SIEM platforms to collect and centralize security logs across heterogeneous sources for analysis and incident detection. [1] Recent studies report alert fatigue caused by large volumes of low-quality alerts, which increases triage burden and hinders effective response. [2] SOC decision-making therefore continues to rely heavily on human analysts and careful automation design. [3] This environment favors solutions that can prioritize risk, explain why a decision was made, and preserve operational safety during response.

Reasoning is therefore positioned as a practical requirement rather than a research luxury. A system that can correlate alerts with context, produce explainable scores, and generate constrained response plans can reduce analyst workload without obscuring accountability. The need is amplified by modern infrastructures where assets, identities, and exposures shift rapidly and static rule sets become brittle.

1.2 Problem Statement



Prior work emphasizes that SIEM systems struggle to identify critical incidents effectively because of false positives and alert fatigue. [2] Rule-based correlation also suffers when redundant or overlapping rules generate excessive false alerts, which further degrades analyst performance. [4] At the same time, studies on SOC automation highlight persistent dependence on human analysts and the need to balance automation with oversight. [3]

The core problem addressed in this thesis is the absence of a generic, interoperable reaction layer that can reason over normalized alerts, produce explainable scores and

plans, enforce policy constraints, and provide auditability, while remaining compatible with existing SIEM deployments.

1.3 Research Objectives



The following objectives guide the work:

- Design a layered architecture that separates orchestration from intelligence and allows model switching without disrupting the SIEM pipeline.
- Implement an explainable risk scoring engine that combines evidence features with context multipliers and produces transparent reasoning.
- Implement a structured planning component that produces policy-checked response plans with rollback support.
- Integrate analyst feedback and investigation traces to support online learning and next-step recommendations.
- Demonstrate the approach with an end-to-end runnable prototype and a reasoning-focused interface.

1.4 Thesis Structure



The remainder of this thesis is organized as follows. Chapter 2 reviews background and related work in SIEM, alert correlation, and response automation. Chapter 3 defines the system requirements. Chapter 4 presents the design and architectural decisions. Chapter 5 describes the system architecture. Chapter 6 details the implementation. Chapter 7 explains the dataset pipeline and training approach. Chapter 8 evaluates the system and presents results. Chapter 9 discusses limitations and implications. Chapter 10 concludes and outlines future work.

Background and Related Work

2.1 Cybersecurity Monitoring and Security Operations Centers

2.2 Security Information and Event Management (SIEM)

2.2.1 Log Collection and Normalization

2.2.2 Event Correlation

2.2.3 Alerting and Incident Response

2.3 Limitations of Current SIEM Systems

2.3.1 Rule-Based Detection

2.3.2 Alert Overload and False Positives

2.3.3 Scalability and Data Volume

2.3.4 Dependence on Human Analysts

2.4 Research Gap

System Requirements

3.1 Functional Requirements



The system is required to ingest alerts from external SIEM sources, normalize heterogeneous payloads into a common schema, enrich alerts with context, and produce ranked incident candidates. A scoring component shall generate an explainable risk score and confidence value for each incident, along with reason codes and context multipliers. A planning component shall generate structured response plans that include recommended actions, priorities, and rollback steps. Plans shall be validated against explicit policy rules before execution, and the execution stage shall record action outcomes and errors.

The system shall support a feedback loop where analysts can label outcomes (true positive, false positive, needs more information) and optionally apply a severity override. Feedback events shall update model weights in a bounded, auditable manner and be stored in an append-only log. The system shall track investigation traces and produce next-step recommendations based on prior analyst actions. A user interface shall expose raw alerts, normalized alerts, scoring explanations, plans, policy checks, execution status, and audit logs.

3.2 Non-Functional Requirements



The architecture shall be modular and allow intelligence models to be replaced without disrupting orchestration. Interfaces between layers shall be explicit and versioned. The system shall be runnable as a single project for demonstration purposes and support repeatable experiments. Explanations shall be human-readable and consistent across runs for comparable inputs. The system shall provide reasonable performance for near-real-time triage, with deterministic behavior for testing and reproducibility.

The implementation shall be maintainable, with clear separation of concerns between domain logic, application services, infrastructure, and the user interface. Logging shall be structured and suitable for audit review. Configuration shall be externalized to avoid recompilation for model switching or environment changes.

3.3 Security and Safety Requirements



Security and safety requirements focus on bounded automation, traceability, and controlled execution. Actions proposed by the planner shall be constrained by explicit policy rules, including allow/deny lists, confidence thresholds, and asset criticality constraints, to prevent unsafe or non-compliant responses. High-impact actions shall require approval gates or dual control, and every automated action shall have a defined rollback or compensating action.

The execution layer shall validate input parameters, enforce entity presence checks, and refuse execution when required context is missing or when policy checks fail. Rate limits and idempotency safeguards shall be applied to prevent repeated execution under noisy or duplicated alerts. When confidence is below a defined threshold, or policy approval is not met, the system shall fall back to recommendation-only behavior rather than automation.

Model risk shall be mitigated through bounded updates and safety defaults. Feedback-driven weight updates shall be limited in magnitude and logged in an append-only audit trail. Model outputs shall remain explainable across updates, with reasoning artifacts retained for audit and post-incident review. Drift detection and periodic offline recalibration shall be supported to reduce silent degradation.

Data handling shall enforce least-privilege credentials for connectors and executors, and sensitive fields shall be minimized or redacted where possible. All security decisions, policy evaluations, and execution outcomes shall be logged with timestamps and actor identity. Logs shall be protected against modification to preserve chain-of-custody and support forensic review.

Design and Architecture Decisions

4.1 Design Goals

4.2 Architectural Style Selection

4.3 Separation of Intelligence and Orchestration Layers

4.4 Reasoning and Explainability Design

4.5 Dataset Pipeline Design

4.6 Safety and Fallback Mechanisms

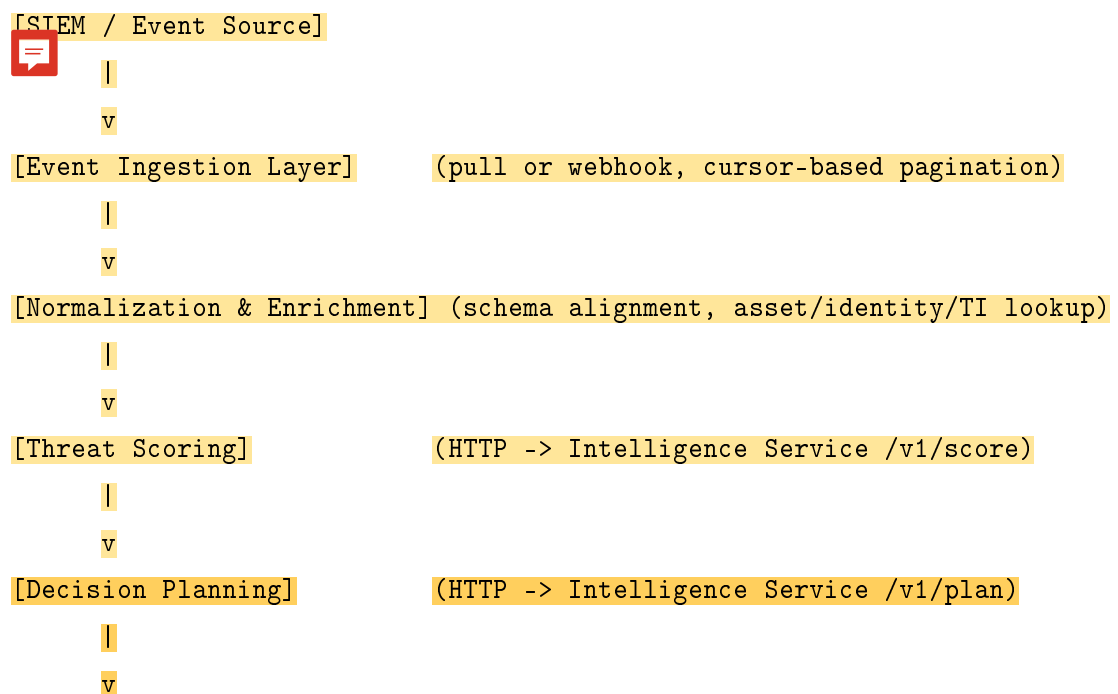
System Architecture

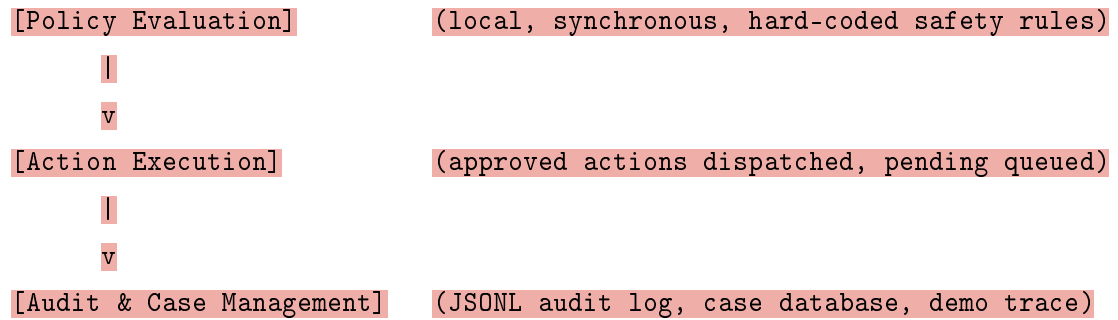
5.1 System Overview



The system is composed of two cooperating processes that communicate over a local HTTP boundary: a **Core Engine** written in C# (.NET 9) and an **Intelligence Service** written in Python 3.11. This two-process design was a deliberate choice rather than an accident of language preference. Security orchestration involves both latency-critical control-plane decisions—acquiring locks, dispatching actions, updating audit logs—and compute-heavy inference that benefits from the Python machine-learning ecosystem. By keeping these concerns in separate processes, each side can be optimised, tested, and deployed independently without the other carrying its complexity.

The overall data flow through the system follows a linear pipeline for each alert:





Each stage is implemented behind a C# interface, making it straightforward to swap implementations—for example, replacing the live SIEM connector with an in-memory replay connector during testing, or substituting the HTTP-based scorer with a direct local call when the intelligence service runs embedded.

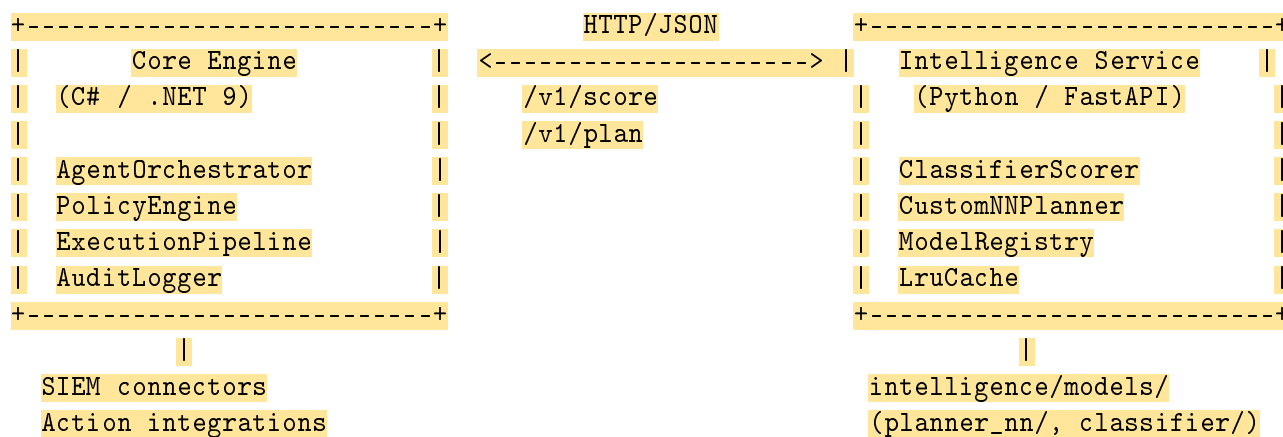


FIGURE 5.1: High-level system decomposition showing the two-process boundary.

The clean HTTP boundary also provides an important operational benefit: the intelligence service can be upgraded—swapping a new model checkpoint or changing the active profile— without restarting the core engine, and vice versa.

5.2 Event Ingestion Layer



Every **SOAR** cycle begins with the ingestion of raw security events. The system supports two ingestion modes, both expressed through the same `ISiemConnector` interface.

Polling mode is the default. The orchestrator calls `PullAlertsAsync` on each cycle, passing a *cursor* that encodes the position reached in the last successful fetch. Cursor-based pagination is stateless from the SIEM's perspective and resilient to process restarts: because the cursor is persisted in memory and re-used across cycles, replaying from the last checkpoint is automatic. When no cursor exists (first run), the engine falls back

to a configurable `SinceUtc` timestamp so that it does not process the entire historical backlog.

Webhook mode is the alternative for SIEMs that push events rather than expose a pull API. A lightweight ASP.NET Core endpoint accepts inbound HTTP POST requests and hands each payload directly to `AgentOrchestrator.HandleAlertAsync`, which is thread-safe and can process concurrent alerts independently. The two modes share all downstream pipeline stages; only the entry point differs.

The `RawAlert` type produced at this stage is intentionally minimal—it carries only the data the SIEM itself supplies: an identifier, a timestamp, a rule name, an alert type, the originating SIEM name, and an opaque `Properties` dictionary for any additional vendor-specific fields. This prevents upstream SIEM schema variations from leaking into downstream components.

5.3 Normalization and Enrichment



Raw alerts arrive in whatever schema the upstream SIEM happens to produce. Before any intelligence component can operate on them, they must be translated into the system's canonical `EnrichedAlert` representation. This is the responsibility of the `INormalizationPipeline` component, implemented concretely as `BasicAlertValidator`.

Schema normalization maps vendor-specific field names to a stable internal schema. Alert type codes are canonicalized to one of the system's five recognized categories: `PortScanFromIp`, `BruteForceUser`, `MalwareHashOnHost`, `SuspiciousProcessOnHost`, and `BenignNoise`. Severity values from heterogeneous scales (0–10, 0–255, Low/Medium/High) are rescaled to a uniform 0–100 integer. Any alert that cannot be mapped to a recognized type is flagged with a `sentinel` value so that downstream components can apply conservative handling.

Context enrichment augments each normalized alert with three independent information lookups performed in parallel:

- **Asset context:** Given a host identifier or IP address extracted from the alert's property bag, the enrichment layer queries an internal asset registry to retrieve the asset's `criticality` (1–5), its network `environment` (internal/DMZ/external), whether it is a privileged host, and any associated tags. Criticality 5 assets trigger mandatory escalation regardless of the scorer's output.
- **Identity context:** If the alert references a username, the identity provider is queried for the user's `privileged` flag, department, and role. Privileged users

(service accounts, administrators) carry higher inherent risk weight in both scoring and planning.

- **Threat intelligence:** IP addresses and file hashes extracted from the alert are checked against a threat-intelligence feed. The result includes a **reputationScore** (0–100) and a list of matching threat-intelligence tags such as *known-scanner*, *c2-server*, or *malware-family*. A high TI reputation score directly amplifies confidence in the scorer output.

All three lookups are resolved before the alert proceeds, ensuring that every downstream component operates on a fully enriched view. If any lookup fails, the enrichment stage records a null for that context block rather than aborting processing—a deliberate fail-open design that prioritises pipeline continuity over completeness.

5.4 Threat Scoring Component

Once an alert is enriched, the orchestrator submits it to the intelligence service’s `/v1/score` endpoint. The returned **ThreatAssessment** contains four fields: a **severity** integer (0–100), a **confidence** floating-point value (0–1), a natural-language **hypothesis**, and a list of **evidence** strings.

The intelligence service scores alerts using a configurable scorer hierarchy. In production mode the default scorer is a **hybrid**: a lightweight **TF-IDF** + Logistic Regression classifier operates as the primary signal, while an optional Ollama-hosted LLM provides a secondary explanation when confidence is borderline. A rule-based fallback ensures the pipeline never returns an empty score even if both primary paths fail.

The **ClassifierScorer** is the workhorse of this hierarchy. It represents alert fields as a bag-of-words TF-IDF vector concatenated with a small set of structured features (alert type, asset criticality, privileged flag, exposure zone). A scikit-learn logistic regression classifier, trained on 8,644 labeled examples, maps this vector to a severity bucket. Post-processing applies calibrated confidence floors per alert type to prevent over-confident low-severity outputs, and boosts the final score when the enrichment context contains a high threat-intelligence reputation or a criticality-5 asset.

The scorer is intentionally decoupled from the planner: it is a read-only assessment that provides evidence for the planning decision, not a trigger. This is important for explainability—the assessment’s **hypothesis** and **evidence** fields are recorded verbatim in the audit log alongside every action taken, so that analysts reviewing historical cases can reconstruct why the system acted as it did.

5.5 Decision Planning Component



Given an enriched alert and a threat assessment, the planner must produce a **DecisionPlan**—a structured object specifying a **strategy**, a **priority**, an ordered list of **planned actions**, and a set of **rationale** strings explaining the choices.

The strategy is one of five values that capture the intended response posture:

- **Ignore**: The alert is assessed as benign noise; no action is taken.
- **NotifyOnly**: Severity is low or confidence is low; alert the SOC and open a ticket.
- **Investigate**: The alert warrants investigation but no automated containment.
- **ContainAndCollect**: Active threat confirmed; containment plus forensic collection.
- **Contain**: Immediate high-confidence threat; containment is the priority.

The intelligence service exposes planning through the `/v1/plan` endpoint. Multiple planner backends are supported and selected via the `INTEL_PLANNER_MODE` environment variable or through a named profile in the model registry. In production, the system uses the **CustomNNPlanner**—a lightweight multi-head MLP trained from scratch on the same 8,644-example dataset as the scorer.

The custom neural network decomposes planning into three parallel classification problems: strategy prediction (5-class softmax), action set prediction (multi-label sigmoid over 8 containable action types), and priority regression (sigmoid scaled to 0–100). This structured decomposition avoids the hallucination and output-format compliance issues inherent in generating free-form JSON with a large language model. Once the model outputs a strategy and action set, a deterministic post-processing pass resolves entity parameters from the enriched alert context (host IDs, IP addresses, file hashes), adds safe infrastructure actions (ticket opening, notifications) that are always appropriate, and applies label sanitization rules to enforce coherence—for example, **IsolateHost** is suppressed when no host identifier is present in the alert.

The model registry allows multiple named profiles to coexist in a single deployment. A caller can request a specific profile by passing the **X-Intel-Profile** HTTP header, enabling A/B evaluation or graceful rollback to the rule-based planner without any configuration change to the core engine.

5.6 Policy and Safety Layer



The policy engine is the system's safety backstop. It operates entirely within the core engine—it never makes external calls—and evaluates every planned action against a set of hard constraints before any action is dispatched. The result is a `PolicyDecision` that triages each planned action into one of three categories:

- **Approved:** The action passes all policy constraints and is auto-executed.
- **Pending:** The action is valid but requires human confirmation before execution. Pending actions are held in an `ApprovalWorkflow` queue and can be approved or rejected via the API.
- **Denied:** The action violates a hard constraint and is permanently blocked for this alert cycle.

Hard denials are evaluated first, before approval gating. They include: actions that require an entity (host ID, IP address) that is not present in the enriched alert; irreversible actions on production assets below a severity threshold; and any action that the system's action catalog has marked as globally disabled. This ordering is important: a pending action that would be denied for missing entity data must be denied immediately rather than queued for human approval, since no human action can supply the missing data.

The threshold-based approval gating layer sits below the hard denials. Actions with a risk score above a configurable threshold require human approval even if all entity constraints are satisfied. The threshold is tunable per environment—in a staging environment, actions may be auto-approved at lower thresholds for testing purposes; in production, the thresholds are set conservatively. All policy decisions, including their specific reasons, are recorded in the audit log and surfaced in the console output during demo mode.

5.7 Action Execution and Response



Approved actions are dispatched by the `IExecutionPipeline` component. Each `PlannedAction` carries a type (e.g., `BlockIp`, `IsolateHost`, `QuarantineFile`), a set of typed parameters, a risk level, and an `expectedImpact` score that the executor uses to prioritize ordering within a plan.

The execution pipeline supports two key operational modes:

Dry-run mode is the default for new deployments. Every action is validated and logged with a **DryRun** status but no external integration is called. This allows teams to observe system behavior in full detail—including which actions would have been executed, with what parameters, and in what order—before committing to live execution. The audit trail produced in dry-run mode is structurally identical to that produced in live mode, so the transition requires no changes to reporting or observability tooling.

Live mode dispatches actions to real integration targets. Action handlers are resolved from the **ActionCatalog**, which maps action types to registered handlers. Handlers are invoked with a configurable timeout (**ActionTimeout**, defaulting to 30 seconds) and their results are collected into an **ExecutionReport** that records each action's final status: **Succeeded**, **Failed**, **Skipped**, or **DryRun**. If **StopOnFailure** is enabled, the execution pipeline halts after the first failed action, preserving system state; otherwise it continues and aggregates all results.

Rollback actions—**UnblockIp**, **UnisolateHost**, **EnableUser**—are included in every plan that contains the corresponding containment action. They are not executed automatically; instead they are stored in the plan's **rollbackActions** list and surfaced to analysts as reversibility documentation. An analyst reviewing a completed case can trigger rollback actions manually via the case management API.

After execution completes, the orchestrator calls **TryAckAsync** to update the SIEM's alert status. Alerts with pending actions are acknowledged as **InProgress**; alerts where all actions succeeded are marked **Closed**; alerts with any failed actions remain **InProgress** for analyst review.

5.8 Audit and Explainability Components

Every significant event in the pipeline is recorded by the **IAuditLogger** interface, which writes structured **AuditEntry** records to a newline-delimited JSON (**.jsonl**) file. Each entry carries a correlation ID, a timestamp, a component name, an event type, a human-readable message, and an open-ended **Data** dictionary for context-specific key-value pairs.

The audit log is append-only and designed to be queryable post-hoc. A companion **IAuditPipeline** component runs after each orchestration cycle to aggregate recent entries into a structured report, providing a rolling summary of system activity that can be consumed by a monitoring dashboard.

In addition to the event-level audit log, the system writes a **demo trace** for each processed alert. A demo trace is a structured snapshot of every stage the alert passed

through: the raw alert, the enriched alert, the threat assessment, the decision plan, the policy decision, and the execution report. These traces are written to a `demo_trace.jsonl` file and serve two purposes. First, they provide complete reproducibility: given a trace file, any alert can be replayed through the pipeline deterministically, making debugging straightforward. Second, they power the demo user interface, which visualizes the full decision pathway for each alert in a form suitable for stakeholder demonstrations.

The `ICaseManager` component provides a higher-level abstraction above individual alert traces. It groups related alerts into *cases* with a unified timeline, aggregating the assessments and plans associated with each case into a single record in a SQLite database. Cases persist across orchestrator restarts and can be retrieved, annotated, and closed via a REST API. This case-level view is particularly valuable for multi-stage attacks where the individual alert-level records need to be joined to understand the full incident.

5.9 Cross-Cutting Concerns

Several concerns span multiple pipeline stages and are worth treating explicitly.

Fault isolation. Each pipeline stage is wrapped in a try-catch boundary within `HandleAlertInternal`. A failure in normalization, planning, policy, or execution produces a structured error record in the audit log and a graceful outcome report, but does not propagate exceptions to the caller or interfere with other alerts being processed concurrently. Audit and case management failures are additionally isolated inside their own try-catch blocks because secondary concerns must never degrade primary alert handling throughput.

Observability. Every alert that completes processing triggers a call to `PrintDemoSummary`, which emits a structured, ANSI-colored console block showing the alert identity, enrichment context, assessment scores, plan strategy, and per-action policy decisions. This output is designed to be human-readable at a glance during live demonstrations and is suppressed automatically when output is redirected (detected via `Console.IsOutputRedirected`). At the end of a session, `PrintExecutiveSummary` emits an aggregate summary table: total alerts processed, strategy breakdown, action counts by approval tier, and the highest-severity alert seen.

Response caching. The intelligence service maintains an LRU cache keyed on a hash of the alert content and the active model profile. Repeated submissions of the same alert—which occur naturally when a SIEM re-delivers unacknowledged events—are served from cache without invoking the scorer or planner, reducing both latency and model inference cost. Cache capacity is configurable via `INTEL_CACHE_SIZE` and defaults to 256 entries.

Configuration. All runtime parameters are expressed through environment variables loaded from a `.env` file at startup. The C# side exposes an `OrchestratorConfig` record with sensible defaults for development use; the Python side reads variables with the `os.environ` API, also with defaults. Neither side requires file system writes or database migrations for configuration changes—a property that simplifies deployment in containerized environments and makes it easy to reproduce exact configurations from a single `.env` snapshot.

Implementation

Building a system that reasons about security threats is, at its core, an exercise in managing complexity across layers that evolve at very different speeds. The orchestration layer must be rock-solid and deterministic — it cannot afford to crash when a model fails or a network call times out. The intelligence layer, on the other hand, needs to be fluid: models get retrained, thresholds get tuned, and entirely new backends can be swapped in overnight. Getting this boundary right was one of the first and most consequential decisions in the implementation.

This chapter walks through every major component of the system, explaining not just what was built but why each decision was made in the way it was. The intent is to give a software engineer enough context to understand, reproduce, or extend any part of the implementation without needing to reverse-engineer intent from code alone.

6.1 Technology Stack

The system is split into two cooperating services with distinct runtime requirements, and the technology choices reflect that distinction honestly.

6.1.1 Orchestration Layer: .NET 9 / C#

The orchestration engine is implemented in C# targeting `net9.0`. The choice of .NET over alternatives such as Java or Go comes down to three practical qualities. First, C#'s type system — particularly its support for sealed records, interfaces, and pattern matching — maps naturally to the domain model. An alert pipeline built around discriminated types like `RawAlert`, `NormalizedAlert`, and `EnrichedAlert` is easier to reason about when the compiler catches contract violations. Second, .NET's native dependency injection container is lightweight and expressive enough for this project without reaching

for an external framework, which reduces the surface area of the system. Third, the `async/await` infrastructure handles the naturally I/O-bound nature of alert processing — connector polling, HTTP calls to the intelligence service, and audit log writes — without requiring explicit thread management.

A deliberate choice was made not to use a full web application framework like ASP.NET for the orchestration engine. The engine runs as a console application, not a web server. This is appropriate because its job is to process alerts, not to serve HTTP requests. The webhook *listener* is a small HTTP utility used only in webhook mode — the engine remains a pipeline, not a service.

6.1.2 Intelligence Layer: Python / FastAPI

The intelligence service is implemented as a Python `FastAPI` application. Python was the only reasonable choice here: PyTorch, scikit-learn, and the Hugging Face ecosystem are Python-first and would have required awkward bridging had the intelligence layer been implemented in another language. FastAPI was chosen over Flask or Django for two reasons. Its `async` request handling matches the expected load pattern (individual scoring and planning requests, not bulk data processing), and its automatic OpenAPI documentation makes the contract between the two services immediately visible during development. There is also a practical benefit: FastAPI's request parsing raises clear validation errors when the payload does not match the expected shape, which surfaces interface mismatches early.

6.1.3 Model Training: PyTorch and scikit-learn

Model training uses PyTorch for the neural planner and scikit-learn for the classifier-based scorer. The two frameworks complement each other well. scikit-learn's pipeline abstraction is ideal for the text-based classifier (TF-IDF vectorization followed by a gradient-boosted classifier), while PyTorch handles the more structured multi-head MLP for planning. Both produce artefacts that can be loaded at inference time without framework-specific servers — `.pt` files for PyTorch and `.joblib` files for scikit-learn — which kept the intelligence service's runtime dependencies minimal.

6.1.4 Persistence

Persistence is intentionally lightweight. Audit events are written to an append-only JSONL file (`data/audit.jsonl`), chosen because JSONL requires no schema migration,

is human-readable, and can be tailed in real time during a demo. Cases — which represent the ongoing state of an investigated incident — are stored in SQLite via the `SQLiteCaseManager`. SQLite was preferred over a full relational database because it requires no server process and runs on any machine, which is appropriate for a single-machine demonstration system.

6.1.5 Configuration



All configuration is driven by environment variables, loaded from a `.env` file at startup by `EnvFileLoader`. This separates code from operational settings — switching from the custom neural planner to the seq2seq planner, for instance, requires only a one-line change to `.env` without recompiling anything. This pattern also makes the system straightforward to reconfigure between different demo scenarios.

6.2 Core Engine Implementation

The orchestration engine is the beating heart of the system. Every alert that enters the pipeline passes through a deterministic sequence of stages, each isolated behind an interface, each recorded in the audit log. The design goal was that any single stage could fail loudly without corrupting the rest — and that an engineer reading the main processing loop should be able to understand the entire system at a glance.

6.2.1 The Orchestration Pipeline

The central class is `AgentOrchestrator`. It holds references to every stage of the pipeline as interface-typed dependencies, injected at construction time via the standard Microsoft DI container. This means that in tests, any stage can be substituted with a stub, and in production, the full implementation is wired up once in `Program.cs` and left alone. The primary processing method, simplified to its essential structure, reads as follows:

```
// Pull a batch of raw alerts from the SIEM connector
RawAlert[] alerts = await connector.PullAlertsAsync(request, ct);

foreach (RawAlert raw in alerts)
{
    // Stage 1: normalize and enrich
    EnrichedAlert enriched = await normalization.ProcessAsync(raw, ct);
```

```
// Stage 2: score the threat
ThreatAssessment assessment = scorer.Score(enriched);

// Stage 3: generate a response plan
DecisionPlan plan = planner.Plan(enriched, assessment, context);

// Stage 4: evaluate plan against policy
PolicyDecision decision = policy.Evaluate(plan, assessment, enriched, context);

// Stage 5: execute approved actions
ExecutionReport report = await execution.ExecuteAsync(
    enriched, assessment, plan, decision, execContext, ct);

// Stage 6: persist audit record and update case
audit.Log(raw, enriched, assessment, plan, decision, report);
caseManager.OpenOrUpdate(assessment, plan, decision);
}
```

What is worth emphasising here is that each stage boundary is an exception boundary too. If normalization fails, the pipeline records a **NormalizationFailed** audit event and moves to the next alert rather than terminating. If planning fails, the same principle applies. Scoring is further protected by a **ScoreSafe** wrapper that catches exceptions and returns a low-confidence fallback assessment, because a scoring failure should never block the rest of the pipeline — a degraded score is better than a dropped alert.

This design was deliberately chosen over an alternative where failures terminate the run. In a real SOC environment, partial failures are normal: a model might time out on one alert while processing the next one perfectly. Failing the entire batch over one bad alert would be operationally unacceptable.

6.2.2 Interface-Driven Architecture

Every major component is defined as a C# interface before it is implemented. The full set of core interfaces is:

- **ISiemConnector**: Abstracts alert sources. Implements **ConnectAsync**, **PullAlertsAsync**, **AckAsync**, and **GetHealthAsync**. Both the Wazuh HTTP connector and the in-memory simulator implement this interface, so the orchestrator does not know which one it is talking to.
- **INormalizationPipeline**: Accepts a **RawAlert** and returns an **EnrichedAlert**. Internally chains validation, field mapping, and enrichment, but the orchestrator sees only one method call.
- **IThreatScorer**: Accepts an **EnrichedAlert** and returns a **ThreatAssessment**. In production this is an HTTP client that calls the Python intelligence service; in tests it is a stub that returns predetermined values.
- **IPlanner**: Accepts an **EnrichedAlert**, a **ThreatAssessment**, and a **PlanningContext**, and returns a **DecisionPlan**.
- **IExecutionPipeline**: Routes approved actions to the correct executor and returns an **ExecutionReport**.
- **IAuditLogger**: Accepts structured **AuditEntry** records and persists them.
- **ICaseManager**: Manages the lifecycle of investigated incidents as persistent cases.

This level of interface coverage may appear over-engineered for a research prototype, but it serves a real purpose: it makes the system testable without external infrastructure, and it makes it straightforward to swap any single component without disturbing the others. The policy engine and approval workflow, for example, went through several revisions — their callers never changed.

6.2.3 Domain Model: Alert Lifecycle

An alert passes through a well-defined lifecycle of types, each richer than the last:

RawAlert

```

→ (normalization) → NormalizedAlert
→ (enrichment)    → EnrichedAlert
→ (scoring)       → + ThreatAssessment
→ (planning)      → + DecisionPlan
→ (policy)        → + PolicyDecision
→ (execution)     → + ExecutionReport

```

The **RawAlert** is the unprocessed SIEM payload — it contains whatever fields the source provided, plus a canonical **AlertId**, **SiemName**, and **TimestampUtc**. The **NormalizedAlert** applies a SIEM-specific mapping registry to extract a common set of fields: **AlertType**, **Severity**, and a structured **Entities** object containing fields like **SrcIp**, **Hostname**, and **Username**. The **EnrichedAlert** layers context on top — an **AssetContext** with asset criticality and environment, a **ThreatIntelContext** with reputation scores and matched tags, and an **IdentityContext** with privilege level.

These types are *C#* records — immutable value objects that are created once and never mutated. Immutability was chosen deliberately: it means that any stage receiving an alert is guaranteed to see the same data that was passed into it, with no risk of a side effect in one part of the pipeline silently changing what another part sees.

6.2.4 Normalization and the Multi-SIEM Mapping Registry

One of the more interesting engineering challenges was handling alerts from heterogeneous SIEM sources. A Wazuh alert structures its fields very differently from a Microsoft Sentinel alert, yet both need to produce a **NormalizedAlert** with the same set of fields. The solution is a **MultiSiemMappingRegistry** that reads the **X-Siem-Name** header from incoming webhook requests and delegates to the appropriate SIEM-specific mapper.

Each mapper knows, for instance, that a Wazuh alert stores the agent hostname at **agent.name** and the attacker IP at **data.srcip**, while a Sentinel alert stores the hostname in **entities[].HostName** and the tactic in **tactics[0]**. The alert type is inferred from the rule name and tactic when an explicit type field is not present. A Sentinel alert with tactic **CredentialAccess** and title containing “brute force” maps to the **BruteForceUser** alert type.

6.2.5 Enrichment Pipeline

After normalization, the enrichment stage consults up to three data providers, each loaded from a JSON file at startup:

- **AssetInventoryEnricher**: Matches **Hostname** or **HostId** against an assets registry. Returns asset criticality (1–5), environment (**prod/dev/dmz**), and asset category.
- **ThreatIntelEnricher**: Matches **SrcIp** and **FileHash** against a threat intelligence registry. Returns reputation score (0–100) and matched tags such as **tor-exit-node** or **credential-stuffing**.

- **IdentityEnricher**: Matches **Username** against an identity registry. Returns privilege level, department, and user risk score.

The enrichment providers are loaded lazily from disk and consulted in order. The **DefaultEnrichmentMerger** then folds their results into the **AlertContext**, combining fields from multiple providers into a single coherent context block. This context block is what flows into the Python intelligence service as part of the scoring and planning payload.

The validation step, implemented in **BasicAlertValidator**, runs before enrichment. It enforces a small set of invariants: the alert must have a non-empty **AlertId**, a non-empty **SourceSiem**, and at least one actionable entity (a host, user, or IP address). Alerts that fail validation are rejected with a structured error rather than silently dropped. This “fail loudly” philosophy prevents the downstream components from operating on underspecified inputs.

6.2.6 Webhook Mode and Polling Mode

The engine supports two operating modes, selected at startup. In **polling mode**, the engine calls **PullAlertsAsync** in a loop against a Wazuh API connector, sleeping for a configurable interval between cycles. In **webhook mode**, a lightweight HTTP listener (**WebhookAlertListener**) binds to a local port and calls **HandleAlertAsync** for each incoming POST request, passing the decoded JSON payload as a **RawAlert**.

The webhook mode was the one used for the demo. Alerts are sent by an external replay script (**scripts/replay_alerts.py**), which reads pre-recorded alert JSON files and POSTs them to the listener with a configurable delay between alerts. This gives full control over the demo pacing while exercising the same code path that real SIEM webhook integrations would use.

6.3 Intelligence Service

The intelligence service is where the cognitive work happens. It exposes two endpoints — **POST /v1/score** and **POST /v1/plan** — and behind each one lives a configurable backend that can be a rule engine, a trained classifier, a neural network, a seq2seq model, an Ollama-hosted LLM, or any combination of these. From the C# engine’s perspective, all of these look the same: send JSON, receive JSON.

6.3.1 Service Startup and Backend Initialisation

When FastAPI starts, a single `_build_services()` function is called to instantiate the scorer and planner backends. This function reads environment variables (or, if a model registry file is present, the active profile from that file) and constructs the appropriate scorer and planner objects. The rationale for doing this at startup, rather than lazily per request, is that model loading is expensive: a seq2seq model or a neural network takes several seconds to load from disk. Failing at startup rather than on the first request is the better operational choice — it is far easier to diagnose a missing model file from a startup error than from a 500 response on the first real alert.

The construction logic, simplified to its structure, looks like this:

```
# At startup:
if planner_mode == "custom_nn":
    planner = CustomNNPlanner(model_path)
elif planner_mode == "seq2seq":
    planner = Seq2SeqPlanner(model_path, max_new_tokens=384, num_beams=4)
elif planner_mode == "ollama":
    planner = OllamaPlanner(base_url, model_name, timeout=120)
elif planner_mode == "hybrid":
    planner = HybridPlanner(local_planner, remote_planner)
else:
    planner = RulePlanner()    # Always-available fallback

# Similarly for scorer:
if scorer_mode == "classifier":
    scorer = ClassifierScorer(model_path)
elif scorer_mode == "hybrid":
    scorer = HybridScorer(classifier, ollama_scorer, fallback=RuleScorer())
```

A `RulePlanner` and a `RuleScorer` are always available as last-resort fallbacks. If a model fails to load, or if an LLM endpoint is unreachable, the system degrades gracefully to rule-based behaviour rather than refusing to start.

6.3.2 Request Handling and the LRU Cache

Each POST to `/v1/score` extracts the alert payload, resolves the active model profile, and checks a fixed-size LRU cache (default: 256 entries) before calling the scorer. The

cache key is a deterministic JSON serialisation of the alert and the profile name. In practice, many demo alerts are identical or near-identical (for instance, a replay of the same event with a different timestamp but the same entities), so the cache significantly reduces redundant model inference.

If the scorer returns **None** — which indicates that the backend could not produce a result, for instance because an LLM timed out — the service falls back to the **RuleScorer** rather than returning an error. This decision reflects a deliberate API contract: the `/v1/score` endpoint always returns a valid assessment, even if that assessment was produced by a fallback. The caller does not need to handle scoring failures.

```
@app.post("/v1/score")
async def score(payload, request):
    alert = payload.get("alert") or payload
    key    = cache_key(alert, active_profile)

    if cached := cache.get(key):
        return cached

    result = scorer.score(alert)
    if result is None:
        result = RuleScorer().score(alert) # guaranteed non-None

    cache.set(key, result)
    return result
```

Planning requests follow the same pattern but without caching, because plans depend on the full assessment context which may vary even for identical alert payloads. If planning raises an exception, the endpoint catches it and returns the output of the **RulePlanner** instead.

6.4 Model Registry and Profile-Based Backend Selection

One of the more sophisticated infrastructure components in the intelligence service is the model registry. The registry is a JSON file (`intelligence/models/registry.json`) that defines named *profiles*, each specifying a scorer type and a planner type with their respective configuration. An example profile looks like:

```
{
  "active": "custom-nn",
  "profiles": {
    "custom-nn": {
      "scorer": {
        "type": "classifier",
        "model_path": "intelligence/models/scorer_classifier"
      },
      "planner": {
        "type": "custom_nn",
        "model_path": "intelligence/models/planner_nn",
        "action_threshold": 0.50
      }
    },
    "seq2seq": {
      "scorer": { "type": "classifier", ... },
      "planner": { "type": "seq2seq", "model_path": "...", "num_beams": 4 }
    },
    "rule-only": {
      "scorer": { "type": "rule" },
      "planner": { "type": "rule" }
    }
  }
}
```

The `ModelRegistry` class reads this file at startup and builds `ModelProfile` objects from each entry. Scorer and planner instances are *lazily* instantiated on the first request for each profile and then cached — loading a 250M-parameter seq2seq model on every request would be unworkable. The active profile can be overridden per request via either the `modelProfile` field in the JSON payload or the `X-Intel-Profile` HTTP header, which makes it straightforward to compare profiles side-by-side without restarting the service.

This design — a declarative registry of named profiles, with lazy instantiation and per-request override capability — was preferred over a simpler approach of hardcoding a single model at startup because it makes model comparison a configuration operation rather than a code change. During development, switching between the custom NN planner, the seq2seq planner, and the rule baseline required only changing a single environment variable.

6.5 Threat Scoring Component

The scorer's job is to answer a deceptively simple question: given everything we know about this alert, how serious is it, and why? The answer is a **ThreatAssessment** containing a **severity** (0–100), a **confidence** (0–1), a natural-language **hypothesis** string, and a list of structured **evidence** items.

6.5.1 The Classifier Scorer

The primary scorer in the deployed configuration is the **ClassifierScorer**. It wraps a scikit-learn pipeline trained on a prompt-based representation of alerts — a text string encoding the alert type, severity, confidence, entities, and assessment context into a single paragraph. The classifier predicts one of five severity labels: **benign**, **low**, **medium**, **high**, or **critical**. The label is mapped to a numeric severity score via a calibrated class-to-severity table.

A key implementation detail is how raw classifier probability is converted to a meaningful confidence value. In a five-class problem, raw **predict_proba** values underestimate decisiveness: a top class probability of 0.55 beating the next class at 0.15 is, in practice, a very decisive classification, but reporting 0.55 as the confidence would look weak to a downstream consumer. To address this, label-calibrated confidence floors are applied when the top class beats the runner-up by at least a $1.1\times$ margin:

```
# Label-calibrated confidence floors
_LABEL_CONFIDENCE_FLOOR = {
    "critical": 0.88,
    "high":     0.72,
    "medium":   0.52,
    "low":      0.38,
    "benign":   0.22,
}

raw_conf = float(max(predict_proba(prompt)))
decisive  = sorted_proba[0] >= sorted_proba[1] * 1.1
floor     = _LABEL_CONFIDENCE_FLOOR.get(label, 0.5)

if decisive and raw_conf < floor:
    confidence = floor
```

```
else:
    confidence = raw_conf
```

This ensures that a decisive classification of “critical” registers a confidence of at least 88%, which more accurately reflects the model’s certainty and makes downstream policy thresholds (which are also expressed as confidence levels) behave sensibly.

6.5.2 Enrichment-Aware Confidence Boosting

One of the more nuanced scoring decisions involves what to do when external enrichment context strongly corroborates — or escalates — the alert. Consider a brute-force alert from an IP address with a threat intelligence reputation score of 95/100 and a tag of `tor-exit-node`. A raw classifier might score this at 72% confidence because the alert signal is moderately strong. But a security analyst looking at the same alert would feel much more certain: a known TOR exit node attacking a production asset is very unlikely to be a false positive.

The scoring component incorporates this reasoning through a post-classification enrichment boost:

```
# Threat intelligence corroboration
if ti_reputation >= 90:
    confidence = max(confidence, 0.90) # near-certain TI match
    severity    = max(severity, 85.0)
elif ti_reputation >= 70:
    confidence = max(confidence, 0.82) # strong TI corroboration
    severity    = max(severity, 75.0)
elif ti_reputation >= 50:
    confidence = max(confidence, 0.70) # moderate TI signal

# Asset and identity context
if asset_criticality >= 5:
    confidence = max(confidence, 0.85)
    severity    = max(severity, 80.0)
elif asset_criticality >= 4:
    confidence = max(confidence, 0.78)
    severity    = max(severity, 70.0)

if privileged:
```



```
confidence = max(confidence, 0.75)
```

The `max()` operator is important: the boosts are floors, not absolute assignments. If the classifier already scores at 92% confidence, the TI boost does not change that. The boosts only activate when external context provides stronger evidence than the model’s raw output.

6.5.3 Hypothesis and Evidence Generation

The `hypothesis` field in the assessment is a human-readable natural-language sentence that explains the alert in terms a SOC analyst would recognise. It is generated deterministically from the alert type, entities, and enrichment context — there is no LLM call involved, just carefully constructed template logic. A brute-force alert against a known privileged account, from an IP flagged as a TOR exit node, generates:

“Multiple failed authentication attempts against account ‘root’ (87 failed attempts) suggest brute-force activity. Origin identified in threat intelligence (tor-exit-node, brute-force, credential-stuffing; reputation 95/100). High likelihood of malicious intent. — Target is a critical production asset; compromised account has elevated privileges.”

This is assembled from three independent pieces: the base activity description (parsed from alert type and entity values), the threat intelligence narrative (constructed when TI reputation ≥ 70), and the enrichment suffix (added when asset criticality or privilege level is notable). The deliberate choice to use deterministic templates rather than LLM generation ensures that the hypothesis is reproducible, debuggable, and never hallucinates entity values.

The `evidence` list is similarly constructed: it is a short array of structured key-value strings (e.g., `alert_type=BruteForceUser, src_ip=185.220.101.47, ti_reputation=95/100`) that document what signals contributed to the score. This supports the explainability goal: an analyst reading the evidence list can immediately see why the system scored the alert the way it did.

6.6 Decision Planning Component

Planning is the most intellectually interesting component in the system. Given a scored alert, the planner must decide what to do: which containment strategy to adopt, which

specific actions to recommend, in what priority order, and with which entity parameters. The planner must be both precise and safe — recommending the right action with the wrong parameter (for instance, blocking the wrong IP) is worse than recommending nothing.

6.6.1 Framing Planning as Structured Prediction

A key early design decision was to treat planning as *structured prediction* rather than text generation. The output of the planner is not a paragraph describing what should happen; it is a structured object with typed fields: a **Strategy** enum (one of five values), a list of **PlannedAction** objects (each with a typed **ActionType** and a **Parameters** dictionary), and an integer **Priority**. Action parameters (IP addresses, hostnames, usernames) are populated directly from the entities extracted during normalization — the model never generates entity values, it only decides which actions to include.

This framing was chosen for a clear safety reason: if a model can generate entity values, it can hallucinate them. An LLM asked to produce a blocking action might generate a plausible-looking but incorrect IP address. By restricting the model’s output to structural choices (which action types to include, what strategy to adopt) and filling parameters from the normalized alert, the planner is structurally incapable of hallucinating entity values.

6.6.2 The Custom Neural Network Planner

The primary planner in the deployed system is a custom multi-head MLP (**CustomNNPlanner**) trained entirely from scratch on a dataset of 8,644 structured examples. The decision to build a custom neural network rather than fine-tuning a pretrained language model was motivated by three factors.

First, the task structure is highly regular: five alert types map to five strategies and eight possible action types, drawn from a fixed vocabulary. This is not a general-purpose language understanding problem; it is a structured classification problem with strong domain regularities. A pretrained LLM brings 250 million parameters of general world knowledge that are simply not needed here.

Second, inference speed matters for a real-time pipeline. A 250M-parameter seq2seq model takes 800–1200ms per example on CPU; the custom MLP infers in under 5ms. The custom MLP can also be loaded entirely in RAM on any development machine without a GPU.

Third, the custom MLP is inherently more interpretable. Its 198-dimensional feature vector is fully documented, and each feature has an explicit semantic meaning. The weights of a fine-tuned T5 model, by contrast, are distributed across attention heads and feed-forward layers in ways that resist simple interpretation.

6.6.3 Feature Engineering

The MLP receives a fixed-length feature vector assembled from the alert and its assessment. The full feature vector has 198 dimensions, composed as follows:

- **Alert type one-hot** (5 dims): Encodes the alert type as a one-hot vector over the five recognised types (`PortScanFromIp`, `BruteForceUser`, `MalwareHashOnHost`, `SuspiciousProcessOnHost`, `BenignNoise`).
- **Severity and confidence** (2 dims): Normalised severity ($\in [0, 1]$) and confidence ($\in [0, 1]$) from the assessment.
- **Entity presence flags** (5 dims): Binary flags for the presence of `srcIp`, `hostId`, `username`, `fileHash`, and `processName`. These flags serve a dual purpose: they are features for strategy selection, and they are used to mask infeasible actions during training and inference.
- **Confidence and severity buckets** (8 dims): Discretised bins for confidence (<0.3 , $0.3\text{--}0.6$, $0.6\text{--}0.85$, ≥ 0.85) and severity (<30 , $30\text{--}50$, $50\text{--}70$, ≥ 70). Buckets were added because the MLP's linear layers struggle to learn sharp threshold behaviour from continuous values alone.
- **Interaction features** (10 dims): Element-wise products of the alert type one-hot with the normalised severity and confidence respectively. These capture, for example, that a brute-force alert at high confidence is qualitatively different from a port scan at the same confidence level.
- **Security keyword presence** (27 dims): Binary membership flags for 27 domain-relevant keywords extracted from the hypothesis and evidence text (e.g., `malware`, `brute`, `privilege`, `lateral`, `forensic`). These capture semantic signals that are not captured by the alert type alone.
- **SIEM source one-hot** (3 dims): Encodes the data source (`CIC-IDS2018`, `MITRE ATT&CK`, `Mordor`), which correlates with the distribution of alert types and entity patterns.

- **ATT&CK technique presence** (1 dim): Binary flag for whether a MITRE ATT&CK technique ID is present in the raw payload, which indicates higher-confidence attribution.
- **TF-IDF text features** (128 dims): Bigram TF-IDF weights computed over the concatenated hypothesis and evidence text, fitted on the full training corpus. TF-IDF features capture lexical patterns that span multiple keywords and are complementary to the keyword presence flags.

The architecture of the MLP itself follows a three-layer shared encoder with three independent output heads:

Input (198 dims)

```

→ Linear(198, 512) + LayerNorm + GELU + Dropout(0.3)
→ Linear(512, 256) + LayerNorm + GELU + Dropout(0.2)
→ Linear(256, 128) + LayerNorm + GELU
--> Shared representation (128 dims)
+-- Strategy head: Linear(128, 64) + GELU + Linear(64, 5) --> softmax
+-- Action head:   Linear(128, 64) + GELU + Linear(64, 8) --> sigmoid
\-- Priority head: Linear(128, 32) + GELU + Linear(32, 1) --> sigmoid * 100

```

GELU activations were used in preference to ReLU because they produce smoother gradient flow, which was found empirically to reduce training variance. LayerNorm was added at each shared layer because the feature vector mixes dimensions with very different scales (binary flags alongside TF-IDF weights alongside normalised continuous values), and normalisation prevents the larger-magnitude dimensions from dominating the early layers. The total parameter count is approximately 270,000 — roughly three orders of magnitude smaller than a fine-tuned T5 model.

6.6.4 Multi-Head Training with Masked Action Loss

Training uses three loss functions simultaneously:

```

# Per-batch training step
strategy_loss = CrossEntropyLoss(strategy_logits, strategy_targets)
action_loss   = BCEWithLogitsLoss(action_logits, action_targets,
                                   weight=class_weights) * entity_mask
priority_loss = MSELoss(priority_pred, priority_targets)

```

```
total_loss = 1.0 * strategy_loss
            + 0.5 * action_loss.mean()
            + 0.1 * priority_loss
```

The `entity_mask` in the action loss is particularly important. For each training example, action labels that are infeasible given the available entities are masked to zero in the loss computation. For instance, if an example does not have a source IP, the `BlockIp` loss is zeroed out for that example regardless of what the target label says. Without this masking, the model would be penalised for not predicting `BlockIp` on examples where no IP was present, which would introduce incoherent gradient signal. The masking ensures that the model learns the association between action types and entity availability, not just the association between alert types and action labels.

The strategy loss weight is set to 1.0 (the highest weight) because strategy selection is the highest-level decision — the action selection should follow from the strategy, not lead it. Action loss is weighted at 0.5 and priority at 0.1, reflecting the fact that priority is the least critical output for policy correctness.

6.6.5 Enrichment Post-Processing Layer

The MLP was trained on a dataset that did not include enrichment context (asset criticality, threat intelligence, identity privilege). Rather than retrain the model with additional features — which would require new training data and a longer feature vector — a deterministic post-processing layer was added that applies rule-based corrections to the model output after inference.

The rationale for this design is that enrichment context provides reliable, high-priority signals that should always override the model’s base output when present. A human analyst would reason the same way: if the model says “observe more” for an alert on a criticality-5 production server with a 95/100 TI reputation score and 90% confidence, that recommendation should be escalated, regardless of what the model’s statistical patterns suggest. The post-processing layer encodes this reasoning explicitly:

```
# Strategy escalation for critical assets
if asset_criticality >= 5 and confidence >= 0.70:
    if strategy in ("ObserveMore", "NotifyOnly"):
        strategy = "Contain"
    elif strategy == "Contain":
```

```
strategy = "ContainAndCollect"

# Privileged identity: don't leave at ObserveMore
if privileged and confidence >= 0.65 and strategy == "ObserveMore":
    strategy = "Contain"

# Force forensic collection for critical assets
if asset_criticality >= 4 and confidence >= 0.65:
    if "CollectForensics" not in current_action_types:
        actions.append({"type": "CollectForensics", ...})

# Priority floors by asset criticality
floor = {5: 85, 4: 70, 3: 55}.get(asset_criticality, 0)
if floor and priority < floor:
    priority = floor
```

Every override appends a human-readable note to the plan's **rationale** list, which is surfaced in the console output as [NOTE] lines. This preserves full traceability: an analyst can always see exactly which enrichment signals caused the model output to be modified.

6.7 Policy and Safety Layer

The policy engine is the safety boundary between intelligence and execution. It receives a fully formed **DecisionPlan** and classifies each action into one of three outcomes: **Approved** (execute autonomously), **Pending** (requires human approval), or **Denied** (blocked unconditionally). This classification is the last chance to intercept a harmful recommendation before it reaches the execution layer.

6.7.1 Three-Tier Decision Model

The policy evaluation proceeds in two passes. The first pass checks for hard denials — conditions where no approval path exists:

```
// Hard denial conditions
if (!catalog.TryGet(action.Type, out var def))
    → Denied: "Unknown or unsupported action type."
```

```
if (action is missing required parameters)
    → Denied: "Missing required parameters."

if (action.Type is in ForbiddenActionsByEnvironment[environment])
    → Denied: "Action forbidden in this environment."

if (confidence < MinConfidenceForAutonomy
    && action.Type is not in SafeLowConfidenceActions)
    → Denied: "Confidence below minimum for autonomous action."

if (criticality >= CriticalAssetThreshold
    && action.Type is in ForbidActionsOnCriticalAssets)
    → Denied: "Action forbidden on critical assets."
```

Actions that survive the denial pass then enter the approval gate check:

```
// Approval gate conditions
if (def.RequiresApprovalByDefault)
    → Pending: "Catalog marks action as approval-required."

if (action.Risk >= RiskApprovalThreshold)
    → Pending: "Risk exceeds approval threshold."

if (environment == "prod" && action.Type is in RequireApprovalInProd)
    → Pending: "Requires approval in production."

if (criticality >= CriticalAssetThreshold
    && action.Type is in RequireApprovalOnCriticalAssets)
    → Pending: "Target asset is critical."
```

Actions that pass both checks are returned as Approved. This two-pass structure — hard denials first, approval gates second — means that a policy rule can always guarantee a denial regardless of any other configuration, while approval gating is additive and can be layered on top.

6.7.2 Entity Presence Validation

One detail that is easy to underestimate is that the policy engine independently validates entity presence for every action, even after the planner has already checked it. This redundancy is intentional. The planner runs in the Python intelligence service and communicates over HTTP; the policy engine runs in the C# orchestration layer. Validating parameters in both layers means that a bug in the planner — or a malformed HTTP response — cannot result in an action being executed with missing parameters.

The parameter validation logic uses alias-aware key lookup: for **BlockIp**, the presence of any of **src_ip**, **srcIp**, **source_ip**, or **ip** in the action parameters is accepted. This robustness against naming variation proved important during integration testing, where different SIEM sources used different field name conventions.

6.7.3 Action Catalog

The **ActionCatalog** is a registry of all recognised action types, each described by an **ActionDefinition** that specifies:

- **RequiredParameters**: The set of parameter keys that must be present for the action to be executable.
- **RequiresApprovalByDefault**: Whether the action enters the approval queue by default, regardless of risk/impact scores.
- **DefaultRisk** and **DefaultImpact**: Baseline scores (0–100) used when the planner does not specify explicit values.

IsolateHost and **BlockIp** are configured with **RequiresApprovalByDefault = false** and reasonable default risk scores, meaning they can be auto-executed when confidence and asset criticality conditions are met. **QuarantineFile** and **KillProcess** carry higher default impact scores and are denied on critical assets, reflecting the operational concern that killing a process on a production server may cause service disruption.

6.8 Action Execution and Response

The execution pipeline routes approved actions to the correct executor and records every outcome. Each executor implements the **IActionExecutor** interface, which exposes a

single method: `CanHandleAsync(action)` to determine if this executor handles the given action type, and `ExecuteAsync(action, context)` to carry out the action.

In the deployed configuration, the following executors are registered:

- **FirewallExecutor** (or **HttpFirewallExecutor**): Handles **BlockIp** and **UnblockIp**. In dry-run mode, logs the intended action without making any network call.
- **HostIsolationExecutor**: Handles **IsolateHost** and **UnisolateHost**. In dry-run mode, simulates the isolation response.
- **UserAccessExecutor**: Handles **DisableUser** and **EnableUser**.
- **TicketingExecutor** (or **HttpTicketingExecutor**): Handles **OpenTicket**.
- **NotificationExecutor**: Handles **Notify**.

The **ExecutorRouter** dispatches each action to the first registered executor that claims it, then collects results into an **ExecutionReport**. Actions for which no executor claims responsibility are recorded as failed. In dry-run mode (the default for all demo scenarios), every executor simulates its action — writing the intended operation to the audit log without making any real network or system call. The dry-run flag is propagated from the environment configuration to the **ExecutionContext** and respected by every executor.

The dry-run default was a deliberate safety choice. During development, testing, and demonstration, the system should never make real infrastructure changes. An engineer working on the planner or policy engine should not need to worry about accidentally triggering a firewall rule on a real network. The system’s first posture is always “recommend and record”; real execution requires an explicit opt-in.

6.9 Audit, Traceability, and Console Output

Auditability was treated as a first-class requirement from the beginning. Every stage of the pipeline writes a structured entry to the audit log: normalization failure, scoring completion, planning completion, policy decision, and execution outcome all produce entries with a shared **CorrelationId** that links them to the same alert. This means that for any alert, the full decision trace can be reconstructed by filtering the JSONL audit file on its correlation ID.

6.9.1 Demo Trace Writer

In addition to the audit log, the system can optionally write a structured demo trace file (`data/demo_trace.json`) via the `IDemoTraceWriter` interface. Each trace entry is a complete snapshot of the alert's state at the end of processing: the raw alert, the enriched alert, the assessment, the plan, the policy decision, and the execution report. This file is what the demo dashboard reads to reconstruct the full reasoning chain for display in the UI.

6.9.2 Console Output

The console output during a live demo is designed to communicate the system's reasoning at a glance without requiring the viewer to read JSON files. Each processed alert produces a structured block:

```
-----
ALERT  1a2b3c...  [WAZUH]
Rule : Multiple failed SSH logins (sshd)
Type : BruteForceUser  |  Severity: 85/100
Asset: web-server-prod  criticality=5/5  env=prod
TI   : reputation=95/100  tags=[tor-exit-node, brute-force]
Ident: userId=root  privileged=True  dept=Infrastructure
Score: confidence=90%  severity=85
      "Multiple failed auth attempts against 'root' (87 attempts). ..."
Plan : strategy=ContainAndCollect  priority=85  actions=4
      [NOTE ] Strategy upgraded to ContainAndCollect - critical asset.
      [NOTE ] CollectForensics added - criticality=5/5.
      [NOTE ] Priority floored to 85 - asset criticality=5/5.
      [APPROVED] BlockIp  (185.220.101.47)
      [APPROVED] IsolateHost  (web-server-prod)
      [APPROVED] CollectForensics
      [PENDING ] DisableUser  (root)  -- target identity is privileged.
-----
```

ANSI colour codes are used throughout: red for high severity and denied actions, yellow for pending actions and moderate severity, green for approved actions and high confidence, cyan for structural elements. Colour output is automatically disabled when the terminal does not support it (detected via `Console.IsOutputRedirected` and the

`NO_COLOR` environment variable convention), so the output degrades cleanly to plain text in CI environments or when piped to a file.

At session end (triggered by Ctrl+C), the `PrintExecutiveSummary()` method produces a high-level statistical summary: total alerts processed, breakdown by SIEM source, strategy distribution, action outcome counts (approved / pending / denied), and the highest-severity alert seen during the session. This summary is accumulated in a thread-safe statistics object (`_statsLock` protected) throughout the session and rendered once on exit.

6.10 Deployment and System Setup

6.10.1 Single-Command Startup

The system is designed to start from a single command after configuring the `.env` file. The C# engine reads the `.env` file via `EnvFileLoader` at process startup, then launches the Python intelligence service as a managed child process via `IntelligenceProcessManager`. The manager starts the FastAPI service with `uvicorn`, polls the `/health` endpoint until the service responds, and then signals the C# engine to continue with the rest of startup. This means that an engineer running the demo does not need to start two terminals or remember to launch the intelligence service separately — starting the C# engine is the only required step.

6.10.2 Webhook Demo Setup

The typical demo setup uses the following sequence:

1. Start the C# engine (webhook mode is selected when `ORCHESTRATOR_WEBHOOK_URL` is set in `.env`).
2. The startup banner is printed, confirming the model profile, enrichment provider count, webhook URL, and dry-run mode.
3. Run the replay script: `python scripts/replay_alerts.py -siem demo -delay 2`.
4. The script sends six production-quality enriched alerts (three Wazuh and three Sentinel) with a two-second delay between each.

5. Each alert is processed through the full pipeline and a formatted summary is printed to the console.
6. Press Ctrl+C to terminate and print the session executive summary.

The `-siem demo` mode was added specifically for the professor demonstration. It selects only the enriched, high-quality alerts from the full dataset, skipping the generic synthetic alerts that have no enrichment context and produce less informative output.

6.10.3 Environment Configuration Reference

All behavioural parameters are controlled by environment variables, which are documented here for completeness:

- `ORCHESTRATOR_WEBHOOK_URL`: If set, starts the engine in webhook mode on the specified URL (e.g., `http://localhost:5050/`).
- `ORCHESTRATOR_DRY_RUN`: If `true` (the default), no real infrastructure actions are taken.
- `INTEL_ACTIVE_PROFILE`: Selects the model registry profile (e.g., `custom-nn`, `seq2seq`, `rule-only`).
- `ENRICHMENT_DATA_PATH`: Path to the directory containing `assets.json`, `threat_intel.json`, and `identities.json`.
- `CASE_DB_ENABLE`: If `true`, enables SQLite case persistence.
- `DEMO_TRACE_PATH`: If set, writes the full demo trace to the specified JSON file.

This environment-driven approach means that the same binary can be reconfigured for different scenarios — local development, professor demo, and (hypothetically) production deployment — by changing a single file, not the code.

Dataset Pipeline and Training

One of the most underestimated aspects of building an intelligent security system is the work that happens before any model ever trains: sourcing the right data, normalising it into a consistent representation, deciding what the model should learn, and discovering — often the hard way — that your first attempt at all of the above is wrong. This chapter documents that journey honestly. The training pipeline for this system went through multiple complete redesigns, from an initial assumption that a pretrained language model would “just work,” through fine-tuning experiments with a large security LLM, through seq2seq models, and ultimately to a purpose-built lightweight neural network that outperforms all of its predecessors on the evaluation benchmark. Each step taught something that the next step needed.

7.1 Security Datasets and Their Roles

No single publicly available dataset covers the full space of alert types, entity patterns, and response scenarios that this system needs to handle. The dataset pipeline therefore draws on four distinct sources, each contributing something the others cannot, combined with a synthetic generation layer that fills gaps and enforces coverage balance.

7.1.1 MITRE ATT&CK (STIX)

The MITRE ATT&CK framework provides a structured taxonomy of adversarial tactics, techniques, and procedures. The framework is distributed in STIX 2.1 format, a machine-readable standard for threat intelligence exchange. For this project, the ATT&CK STIX data serves two purposes. First, it provides authoritative labels: the technique ID attached to a simulated alert can be used to confirm that the alert type classification is grounded in real-world adversary behaviour rather than invented categories. Second, the

technique descriptions provide vocabulary for hypothesis text generation — the language used to describe a lateral movement technique in ATT&CK is the same language a SOC analyst would use, which helps the generated training data sound authentic.

The STIX data is downloaded via a dedicated script and parsed using the `stix2` Python library. Relevant technique metadata — ID, name, tactic mapping, and description — is extracted and stored locally for lookup during alert generation.

7.1.2 Mordor / Security Datasets

The Mordor project (now called Security Datasets) provides pre-recorded telemetry from realistic attack emulations run in a controlled lab environment. The datasets contain Sysmon process creation events, registry modifications, network connections, and Windows Event Log entries captured during actual execution of offensive toolkits. This data is valuable because it shows what attacks look like in raw logs — not as abstract technique descriptions, but as concrete process names, command-line arguments, and file paths.

For this project, the Mordor data contributes two things: realistic entity values for the training examples (real hostnames, real process names, real registry paths) and realistic entity *co-occurrence* patterns. For instance, a Mimikatz execution on a domain controller reliably produces both a process creation event and a network connection, which helps the model learn which entity types to expect together for each attack type. These co-occurrence patterns proved critical for teaching the planner when it is appropriate to combine `IsolateHost` with `KillProcess` versus treating them as alternatives.

7.1.3 CIC-IDS2018

The Canadian Institute for Cybersecurity Intrusion Detection System dataset (CIC-IDS2018) is a large labelled network flow dataset containing attacks across seven categories: brute force, heartbleed, botnet, DoS, DDoS, web attacks, and infiltration. The flows are labelled at the packet level, with detailed statistics — packet lengths, inter-arrival times, flag counts — that allow realistic severity calibration.

For the planner training data, CIC-IDS2018 contributes the network-level perspective: port scan alerts naturally associate with source IP entities and `BlockIp` actions; brute force alerts associate with username entities and `DisableUser` actions; the severity distributions by attack category inform the numeric ranges used in synthetic alert generation. The dataset is large (over 16 million flows across 10 days of simulated network traffic), so only the labelled attack flows are used, sampled to avoid class imbalance.

7.1.4 DARPA Transparent Computing (TC)

The DARPA Transparent Computing programme produced provenance-rich datasets capturing multi-stage attack scenarios with full system call traces. Unlike the other datasets, TC data includes temporal dependencies: the provenance graph connects a malicious email attachment to the browser process that opened it, to the child process that spawned, to the registry key it modified, and to the external C2 connection it established. This richer contextual structure informs how the system should reason about alert chains rather than treating each alert as isolated.

For training purposes, the TC data is used to construct malware and suspicious process scenarios that have realistic entity chains — alerts that come with a host ID, a process name, and a file hash, which is the complete entity set needed for a `QuarantineFile` response action. Without this dataset, file hash entities were severely under-represented in training, causing early models to consistently miss the `QuarantineFile` action even when the alert clearly indicated malware.

7.1.5 Synthetic Scenario Generation

Despite the above sources, no combination of real datasets provides sufficient coverage of all five alert types, all eight action types, and all five strategy labels in balanced proportions. Real datasets are naturally imbalanced: port scans and brute force events are common; malware file hash detections with a complete entity set (host, process, hash) are rare. A model trained on this imbalance would learn to over-predict the common cases and under-predict the rare but critical ones.

The synthetic generation layer, implemented in `generate_dataset.py`, addresses this by generating structured training examples algorithmically. For each alert type, a generator function produces a plausible alert with randomised entity values, paired with a scored assessment and a labelled response plan. The generators are not naive random — they encode domain knowledge. For instance, a brute force generator samples a failure count from a realistic distribution (50–500 attempts), assigns a severity in the 60–85 range, and generates a plan that includes `DisableUser` when a username entity is present and `BlockIp` when a source IP is present, falling back to `NotifyOnly` when neither is available:

```
def _gen_brute_force(rng):
    src_ip    = _random_ip(rng)
    username  = rng.choice(["admin", "root", "service_acct", ...])
    attempts  = rng.randint(50, 500)
```

```
has_ip    = rng.random() > 0.3    # 70% of cases have src IP
has_user  = rng.random() > 0.1    # 90% of cases have username

alert = { "type": "BruteForceUser",
          "severity": rng.randint(60, 85),
          "entities": {
              "srcIp":    src_ip if has_ip else None,
              "username": username if has_user else None } }

# Actions conditioned on which entities are actually present
actions = []
if has_ip:    actions.append({"type": "BlockIp",
                             "parameters": {"src_ip": src_ip}})
if has_user: actions.append({"type": "DisableUser",
                             "parameters": {"username": username}})
actions.append({"type": "OpenTicket", "parameters": {}})
```

This entity-conditional generation was not present in the first version of the generator. The early version always generated all entities and always generated all actions, which meant the model could never learn the correct mapping between entity presence and action feasibility. When a `BlockIp` action appeared in every training example regardless of whether a source IP was present, the model learned to predict `BlockIp` unconditionally — and then the policy engine had to reject those actions at runtime because there was no IP to block. This is a concrete example of a data quality problem that looked like a model quality problem until traced back to its source.

7.2 Dataset Processing and Normalisation

Each data source arrives in a different format. STIX is JSON-LD; Mordor is JSON event logs; CIC-IDS2018 is CSV network flows; DARPA TC is compressed provenance graphs. Before any training can happen, all of these must be converted into the same canonical alert schema that the production system uses, so that the model is trained on inputs that are identical in structure to what it will see at inference time.

7.2.1 Schema Mapping

The canonical alert schema used for training mirrors the compact JSON structure that the production `NormalizationPipeline` produces:


```

{
  "type":      "BruteForceUser",
  "severity":  75,
  "sourceSiem": "cic-ids",
  "entities": {
    "srcIp":    "192.168.1.50",
    "username": "admin",
    "hostname": null,
    "fileHash": null,
    "processName": null
  },
  "rawPayload": {
    "attempts":    237,
    "failed_logins": 237,
    "rule_id":     "5710"
  }
}

```

Mapping real data to this schema requires SIEM-specific extractors. A CIC-IDS2018 CSV row with label “Brute Force” is mapped to the **BruteForceUser** type; the source IP from the “Source IP” column maps to **srcIp**; the packet count maps to the **attempts** raw field. A Mordor Sysmon event with Event ID 1 (process creation) maps to **SuspiciousProcessOnHost**; the **Image** field maps to **processName**; the **Computer** field maps to **hostname**.

One decision that turned out to matter significantly: fields that do not exist in the source data (such as **fileHash** for a network flow event) are explicitly set to **null** rather than omitted. A model trained on examples where absent fields are simply missing rather than explicitly null learns a different — and weaker — representation of entity absence. When **null** is explicit, the model learns “this field is present but empty”; when the field is missing, the model cannot distinguish between “not applicable” and “not yet extracted.” Explicit nulls produced noticeably better entity-awareness in downstream planner predictions.

7.2.2 Label Assignment and Severity Calibration

Severity labels for the scorer training data are assigned semi-automatically. For each alert type, a severity range is defined based on domain knowledge and calibrated against the severity distributions in the real datasets:

- **MalwareHashOnHost**: 75–95 (a known-bad file hash is near-definitive evidence)
- **BruteForceUser**: 60–85 (severity scales with failure count and account type)
- **SuspiciousProcessOnHost**: 50–80 (broad range; high false positive rate)
- **PortScanFromIp**: 40–70 (reconnaissance only; rarely immediately harmful)
- **BenignNoise**: 0–20 (must score very low to train the model to suppress false positives)

Within each range, severity is modulated by context: higher failure counts increase brute force severity; a file hash entity matching a known-bad pattern pushes malware severity to the top of its range; alerts from external IP ranges receive higher severity than those from internal subnets.

7.2.3 Action Balancing

A significant problem discovered during early evaluation was that the planner training data was heavily imbalanced in its action labels. `OpenTicket` and `Notify` appeared in nearly every example; containment actions (`BlockIp`, `IsolateHost`, `QuarantineFile`, `KillProcess`) appeared in far fewer. The model learned to default to notification-only responses even for high-severity threats — because the training data had taught it that notification was almost always the right answer.

The fix was a deliberate action-balancing step in the dataset builder. The builder first constructs the full unbalanced dataset, then up-samples examples that contain under-represented containment actions until each action type appears at a minimum frequency. This is different from standard label balancing because one training example can contain multiple actions; the balancing must operate at the action-occurrence level, not the instance level. After balancing, the distribution of containment actions in the final 8,644-example training set reached workable proportions across all eight action types, with containment actions collectively present in over 60% of examples rather than the 20% seen in the unbalanced version.

7.3 Training Data Format Evolution

The format of the training prompt went through three major revisions, each driven by a specific failure mode observed in the model trained on the previous version.

7.3.1 Version 1: Full JSON Prompt

The first format sent the complete alert and assessment as pretty-printed JSON objects:

```
AlertSummary: {
  "type": "BruteForceUser",
  "entities": { "srcIp": "10.0.1.50", "username": "admin", ... },
  ...
}
AssessmentSummary: { "severity": 75, "confidence": 0.80, ... }
Generate a response plan:
```

This format had two problems. First, for seq2seq models, the prompt was 800–1,200 tokens before the response began, leaving little room for the completion within the model’s context window. Second, the model had to parse nested JSON to extract entity values for action parameters — it would reliably hallucinate plausible-looking but incorrect IP addresses rather than copying the exact value from the input.

7.3.2 Version 2: Canonical Flat Format

The second format flattened the alert into a structured but non-JSON representation. This was shorter and clearer, but it still retained a JSON `Entities` block, which required the model to parse JSON-within-text to populate action parameters — still leaving room for hallucination.

7.3.3 Version 3: Compact Pipe-Delimited Format (Final)

The final format, used for the 8,644-example dataset, uses a flat key-value prompt and a pipe-delimited completion:

```
Type: BruteForceUser
Severity: 75.0  Confidence: 0.80  (high)
Entities: {"srcIp": "10.0.1.50", "username": "admin", ...}
Hypothesis: Multiple failed logins from external source.
Evidence: ["src_ip=10.0.1.50", "attempts=237", ...]
Completion: Contain|BlockIp(src_ip=10.0.1.50),DisableUser(username=admin),
            OpenTicket,Notify|priority=80
```

The pipe-delimited completion format was chosen for a specific technical reason that is easy to miss: the `flan-t5-base` tokeniser (SentencePiece) cannot generate the `{` and `}` characters. This makes it structurally impossible to train a T5 model to output JSON. The pipe-delimited format encodes the same information using only characters that SentencePiece can produce. At inference time, a dedicated parser (`parse_pipe_completion`) reconstructs the full structured plan from the pipe-delimited string, then `sanitize_plan` validates it before it is returned to the C# engine.

This is a representative example of a constraint imposed not by the problem but by a tooling choice, a pattern that appeared repeatedly in the training pipeline. Discovering it required actually trying to train on JSON-format completions and noticing that the model's outputs never contained curly braces — then tracing that back to the tokeniser's vocabulary.

7.4 The Scorer: From Non-Functional Baseline to Calibrated Classifier

The scorer converts a normalised alert into a numerical severity score, a confidence value, a hypothesis, and an evidence list. Getting this right took three distinct attempts, each revealing a different class of failure.

7.4.1 Attempt 1: Zero-Shot LLM (`baron-security`)

The first attempt assumed that a security-focused language model, prompted with the alert payload, would produce reasonable risk scores without any task-specific training. The model was `baron-security`, an Ollama-hosted LLM fine-tuned on security content.

The results were unambiguous and uniformly bad. Every alert — port scans, brute force attacks, confirmed malware hash detections, and benign DNS queries — received the exact same response:

severity: 0, confidence: 0, hypothesis: "No malicious activity detected."

Average severity across all six test alerts was 0.0; average confidence was 0.0. No evidence was provided for any alert. The model's instincts toward caution and non-committal safety, which are appropriate for many LLM applications, made it completely useless as a threat scorer. It had learned that the safe answer is always to deny any threat, and nothing in its zero-shot prompt could override that.

This failure was surprising because the assumption had been that a *security-focused* LLM would at minimum recognise that a confirmed malware hash deserved a non-zero severity. The reality was that no amount of prompt engineering fixed it — the model's learned behaviour was too strong. Fine-tuning was required.

7.4.2 Attempt 2: QLoRA Fine-Tuning on Foundation-Sec-8B

The second attempt fine-tuned Foundation-Sec-8B, an 8B-parameter security-focused model, using QLoRA (4-bit NF4 quantisation, LoRA rank 16) on a dataset of 3,270 synthetic prompt-completion pairs:

```
lora_config = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1, task_type="CAUSAL_LM"
)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)
```

Training ran for 2 epochs on an RTX 5070 Ti (16GB VRAM), taking approximately 1 hour 45 minutes. Final training loss was 0.41. The results were dramatically better than the baseline. Malware alerts now scored severity 90, confidence 0.85; port scans scored 40, confidence 0.55; benign noise correctly scored 0, confidence 0.20. The model produced specific, meaningful hypotheses and evidence lists. Threat discrimination — entirely absent before — was now present and qualitatively correct.

However, the LLM approach carried a significant operational cost. Even with 4-bit quantisation, inference took 2–4 seconds per alert. Exporting the fine-tuned LoRA adapter to an Ollama-compatible format added an additional operational step. And the model's behaviour on out-of-distribution alert patterns was difficult to predict or debug, because the relevant decision logic was distributed across billions of weights rather than expressed in inspectable code.

7.4.3 Attempt 3: TF-IDF Classifier Pipeline (Final)

The third and final approach recognised that scoring is fundamentally a classification problem. Given the alert type and context, the severity label (benign / low / medium

/ high / critical) can be predicted by a discriminative classifier trained on the same prompt-completion pairs. The scorer is a scikit-learn pipeline:

```
pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2),
                             max_features=10000,
                             sublinear_tf=True)),
    ("clf", LogisticRegression(max_iter=500,
                              class_weight="balanced", C=1.0))
])
pipeline.fit(train_prompts, train_labels)
joblib.dump(pipeline, output_path / "model.joblib")
```

Training takes under 30 seconds on CPU; the saved `.joblib` artefact is 2.3MB; inference is sub-millisecond. The classifier achieves 75% severity class accuracy on the held-out evaluation set and a severity MAE of 8.30 points on a 0–100 scale. These numbers appear modest, but they are appropriate for the task: the classifier’s role is to produce a severity label that drives confidence calibration and enrichment boosts, not to be the sole determinant of the final score. The TI reputation and asset criticality enrichment boosts in the scoring component modulate the label-derived score significantly for the alerts that matter most.

A key refinement was the **label-calibrated confidence floor**. In a 5-class classification problem, raw `predict_proba` values underestimate decisiveness: a top-class probability of 0.55 beating the runner-up at 0.15 is an extremely decisive prediction relative to the 0.20 random baseline, yet reporting 0.55 as the confidence would cause the policy engine to treat the assessment as uncertain. The calibrated floors apply label-specific minimum confidence values when the classifier makes a decisive prediction (top class $\geq 1.1 \times$ runner-up). A decisive prediction of “critical” registers at least 88% confidence; a decisive “high” registers at least 72%. This makes the downstream confidence thresholds in the policy engine behave as intended.

7.5 The Planner: Four Approaches to Structured Prediction

The planner’s training journey was more complex than the scorer’s, because planning is a harder problem: the output is not a single label but a structured object with multiple interdependent fields that must be internally consistent and entity-grounded.

7.5.1 Attempt 1: Zero-Shot LLM

The same model that failed on scoring also failed on planning, though differently. Rather than returning empty outputs, it defaulted to the same passive response for every alert: `strategy: NotifyOnly, actions: [OpenTicket, Notify], priority: 50`. Every alert — benign DNS query and confirmed malware execution alike — received the same treatment. Containment actions were never suggested. Additionally, the response was wrapped in a nested `"plan": {...}` key that the C# engine's JSON deserializer did not expect, causing runtime failures.

7.5.2 Attempt 2: Foundation-Sec-8B QLoRA Fine-Tuning

Fine-tuning Foundation-Sec-8B on 3,270 planning examples (2 epochs, 2h 40m, final loss 0.24) produced qualitatively correct results: malware alerts generated `ContainAndCollect` strategies with `QuarantineFile` and `CollectForensics` actions; port scans generated `NotifyOnly` with `OpenTicket`. The schema nesting issue was resolved.

However, new failure modes emerged. Two of the six test alerts produced empty plan objects — the model appeared to fall back to an internal “insufficient confidence” shortcut that bypassed plan generation. More seriously, the model hallucinated entity values in action parameters. It would generate a plausible-looking but incorrect IP address for a `BlockIp` action rather than copying the exact `srcIp` value from the input. For an autonomous response system, this is not a minor accuracy problem; it is a safety failure. Blocking the wrong IP because the model invented a plausible-looking address could cause service disruption on a legitimate host.

This failure motivated the architectural insight described in Chapter 6: action parameters must be populated from the extracted alert entities, not generated by the model. The model's job is to decide *which* action types to include; the entity lookup is deterministic and happens after model inference.

7.5.3 Attempt 3: Seq2Seq Fine-Tuning (flan-t5)

The seq2seq approach reformulated planning as text-to-text translation. The input is the compact alert prompt; the output is the pipe-delimited completion string. Because entity values in the output must match values present in the input, and the model is explicitly trained to copy them rather than generate them, hallucination is substantially reduced.

flan-t5-base (250M parameters) was chosen over decoder-only models for a deliberate reason: its encoder-decoder architecture processes the full input before generating any output, encouraging the model to attend to all entity values before committing to a plan. A causal decoder-only model generates left-to-right from the first token, which would require it to produce action parameters before it has “seen” all the entity context in the prompt.

The first training run immediately surfaced a hardware-specific compatibility issue: training with **fp16** mixed precision on an RTX 5070 Ti (Ada Lovelace architecture) produced **NaN** loss from the first training step. Switching to **bfloat16** resolved the issue. This was a non-obvious debugging step because the error output said only that the loss was **NaN** — not why. Isolating the root cause required checking whether the issue was in the model, the data, the loss function, or the hardware-precision combination, and systematically ruling out each:

```
# fp16 causes NaN loss on RTX 5070 Ti / Ada Lovelace
# bf16 is required:
training_args = Seq2SeqTrainingArguments(
    bf16=True,
    fp16=False, # NaN loss if True on Ada Lovelace
    ...
)
```

Multiple dataset variants and training configurations were explored systematically:

- **Canonical dataset, 2 epochs:** strategy accuracy 80%, action F1 77.1% (30 samples). The full JSON prompt format was too long; the model’s effective attention was diluted.
- **Compact dataset, 2 epochs:** strategy accuracy 90%, action F1 91.1% (100 samples). The compact prompt format showed substantial improvement.
- **flan-t5-large, compact:** strategy accuracy 80%, action F1 77.1% (30 samples). Identical to flan-t5-base on the same data — the larger model brought no benefit.
- **Compact dataset, 8 epochs:** **strategy accuracy 96.5%, action F1 97.7%** (200 samples). Additional training epochs produced significant gains; the compact format with extended training became the benchmark.

The flan-t5-large result deserves comment. Doubling the parameter count from 250M to 770M produced no improvement over the base model on the same data. This is a

meaningful finding: it suggests that the performance ceiling at this stage was set by data quality, not model capacity. A larger model with the same training data learns the same patterns as a smaller model, just more redundantly. Improving the data — adding TF-IDF features, fixing entity masks, balancing actions — improved results far more than increasing model size.

7.5.4 Attempt 4: Custom Multi-Head MLP (Final)

The custom MLP planner is described architecturally in Chapter 6. Here the focus is on the training process and what made it work.

The primary training innovation was **masked action loss**. Earlier training datasets implicitly assumed that every action label in the target was feasible for the given input. Many examples included `BlockIp` in the target even when no `srcIp` entity was present, and `QuarantineFile` even when no `fileHash` was available. The BCE loss for these infeasible actions was computed anyway, providing incoherent gradient signal. The fix computes an entity mask for each training example and zeros out the loss for infeasible action slots:

```
# Entity-aware mask: True where the action is feasible given entities
def _build_entity_mask(entities):
    has_ip    = bool(entities.get("srcIp"))
    has_host  = bool(entities.get("hostname") or entities.get("hostId"))
    has_user  = bool(entities.get("username"))
    has_hash  = bool(entities.get("fileHash"))
    has_proc  = bool(entities.get("processName"))
    return torch.tensor([
        has_ip,                # BlockIp
        has_host,              # IsolateHost
        has_user,              # DisableUser
        has_host and has_proc, # KillProcess
        has_host and has_hash, # QuarantineFile
        True,                  # CollectForensics (always feasible)
        True,                  # OpenTicket
        True,                  # Notify
    ], dtype=torch.float32)

# Applied in the training loop:
action_loss = BCEWithLogitsLoss(reduction="none")(action_logits, targets)
```

```
action_loss = (action_loss * entity_mask).mean() # zero out infeasible
```

A second important fix was **label sanitisation**. The training dataset had been built from LLM-generated examples, some of which included rollback actions (**UnisolateHost**, **UnblockIp**, **EnableUser**) in the primary action list rather than the rollback list. These rollback actions are deterministically derived from the primary containment actions and should never be predicted as primary plan actions. Sanitisation strips them from the training targets, ensuring the model never learns to predict them in the wrong position.

TF-IDF bigram features provided the largest single accuracy improvement for the MLP. The initial 70-dimensional structured feature vector captured alert type, severity, confidence, entity flags, buckets, and keyword presence. Adding 128-dimensional TF-IDF features computed over the hypothesis and evidence text increased the feature vector to 198 dimensions. TF-IDF bigrams capture lexical patterns across pairs of adjacent words — for instance, the bigram “lateral movement” signals containment more strongly than either word alone. This expansion produced a jump from approximately 88.5% to 96.4% strategy accuracy on the pipeline evaluation.

7.6 Evaluation and Results Comparison

Table 7.1 summarises the planner evaluation results across all approaches. Evaluation uses two metrics: **strategy accuracy** (exact match on the predicted strategy label against the ground-truth label) and **action F1** (macro-averaged F1 on the predicted action type set, with rollback actions stripped from both predictions and targets before comparison). The pipeline evaluation runs the predicted plan through **sanitize_plan** before scoring, ensuring that the metrics reflect post-sanitisation outputs rather than raw model outputs.

The custom MLP matches the 8-epoch seq2seq model within rounding error on a much larger evaluation set (865 samples versus 200), using 270,000 parameters instead of 250 million, and inferring in under 5 milliseconds instead of 800–1,200 milliseconds. The relaxed strategy accuracy metric (where semantically adjacent strategies such as **Contain** and **ContainAndCollect** count as correct for the stricter target) is 96.8% for the MLP — meaning that the overwhelming majority of remaining errors are minor escalation differences rather than fundamental strategy mistakes.

The most important finding from this progression is that the task is highly learnable from structured features alone. Once the prompt format was corrected, the dataset was balanced, entity masking was applied, and TF-IDF features were added, all models

TABLE 7.1: Planner evaluation results across all training approaches.

Model	Type	S.Acc.	Act.F1	N	Notes
baron-security (baseline)	Ollama LLM	—	—	6	Passive only; no discrimination
Foundation-Sec-8B (fine-tuned)	QLoRA LLM	0.80	0.84	30	Hallucinated entity values
flan-t5-base (canonical, 2ep)	Seq2Seq	0.80	0.77	30	Full JSON prompt too long
flan-t5-base (compact, 2ep)	Seq2Seq	0.90	0.91	100	Compact format gain
flan-t5-large (action-balanced)	Seq2Seq	0.80	0.77	30	No gain over base model
flan-t5-base (compact, 8ep)	Seq2Seq	0.965	0.977	200	Best seq2seq result
Custom MLP (pipeline eval)	MLP	0.964	0.972	865	270K params, <5ms inference

trained on the same data converged to similar performance. The gap between a 250M-parameter pretrained seq2seq model and a 270K-parameter custom MLP is less than one percentage point on both metrics. Performance is bounded by dataset quality, not model capacity, and the next round of improvements should invest in richer training data — more diverse entity patterns, more SIEM sources, more edge cases — rather than larger architectures.

7.7 Integration with the Intelligence Service

Once trained, a model must integrate into the live intelligence service without disrupting existing inference. The model registry described in Chapter 6 is the integration point: adding a new trained model requires only a new profile entry in `registry.json` pointing to the saved artefact directory. No code changes are needed in the service itself.

Each artefact is self-describing. The custom MLP saves three files: `model.pt` (PyTorch state dict), `meta.json` (input dimension, per-action probability thresholds, training date, and dataset version), and `tfidf.pkl` (the fitted TF-IDF vectoriser). The metadata file ensures that the inference code loads a model compatible with its own feature extraction logic: if the feature vector dimension changed between training runs, the mismatch is caught at load time rather than at inference time, where a silent wrong answer would be produced. The seq2seq artefact saves as a Hugging Face checkpoint directory loaded directly by `Seq2SeqPlanner`. The classifier scorer saves `model.joblib` and a `meta.json` with the class-to-severity mapping.

All artefact formats are versioned through `meta.json`, making it possible to retain multiple model generations and switch between them via the registry without rebuilding anything. During the evaluation phase, this made it straightforward to run the same

alert batch through the seq2seq planner and the custom MLP planner back-to-back and compare their outputs directly, which is how the data in Table 7.1 was collected.

Evaluation

8.1 Evaluation Setup

8.2 Detection Performance

8.3 Response Planning Evaluation

8.4 Example Incident Scenarios

8.5 Discussion of Results

Discussion

9.1 Advantages of the Proposed System

9.2 Comparison with Traditional SIEM Approaches

9.3 Limitations

9.4 Threats to Validity

Conclusion and Future Work

10.1 Summary of Contributions

10.2 Future Research Directions

Several directions are recommended to strengthen both scientific validity and operational readiness. First, evaluation should be extended to larger and more diverse real-world datasets, including multi-tenant environments and mixed cloud/on-prem deployments, to test generalization under heterogeneous data distributions. Second, the intelligence layer can be expanded with continual-learning safeguards, including drift detection, bounded weight updates, and periodic offline recalibration to reduce the risk of performance degradation over time.

Third, response planning quality can be improved by incorporating richer context and explicit constraints, such as change-management windows, business process dependencies, and human approval workflows. Formal policy verification and richer rollback semantics would further increase safety for high-impact actions. Fourth, explainability can be strengthened by aligning model explanations with standardized security taxonomies and by validating that explanations remain stable under adversarial or noisy inputs.

Finally, deployment studies with SOC analysts are needed to quantify workflow impact, measure trust calibration, and identify failure modes that are not visible in offline evaluation. Longitudinal user studies would allow the cognitive loop to be assessed under realistic investigation pressure. Together, these directions provide a path toward a robust, adaptive reasoning layer that can be integrated into production SIEM environments without compromising safety or accountability.

References

- [1] Karen Kent and Murugiah Souppaya. Guide to computer security log management. Technical Report NIST SP 800-92, National Institute of Standards and Technology, 09 2006. URL <https://csrc.nist.gov/pubs/sp/800/92/final>.
- [2] Tao Ban, Takeshi Takahashi, Samuel Ndichu, and Daisuke Inoue. Breaking alert fatigue: Ai-assisted siem framework for effective incident response. *Applied Sciences*, 13(11):6610, 2023. doi: 10.3390/app13116610. URL <https://www.mdpi.com/2076-3417/13/11/6610>.
- [3] J. Tilbury and S. Flowerday. Humans and automation: Augmenting security operation centers. *Journal of Cybersecurity and Privacy*, 4(3):388–409, 2024. doi: 10.3390/jcp4030020. URL <https://www.mdpi.com/2624-800X/4/3/20>.
- [4] S. Sarhan, M. Haque, M. Asad, P. Kabo, A. Apon, and I. M. Elfadel. Rulegenie: Siem detection rule set optimization. *arXiv preprint arXiv:2505.06701*, 2025. doi: 10.48550/arXiv.2505.06701. URL <https://arxiv.org/abs/2505.06701>.

Appendix

A.1 Additional Figures

A.2 Configuration Details